

Semester: Summer2026

Term Project Report

Project: Dhaka City Flood-Safe Route Finder using Graph Algorithms

Course Code: CSE 246

Section:01

Submitted to:

- Dr. Md. Tauhid Bin Iqbal
- Assistant Professor
- Department Of Computer Science and Engineering

Submitted by:

Project Team Members	ID
S.M. Taif Chowdhury	2024-3-60-250
Sadiqur Rahman Bhuiyan	2024-3-60-230
Sanjida Alam Mithi	2025-1-60-355
Awandrila Sarker	2025-1-60-365

1. Introduction

1.1 Problem Statement

Dhaka, the capital of Bangladesh, is a densely populated megacity where traffic is a constant obstacle. Monsoon season makes it worse. Heavy rainfall triggers sudden waterlogging and flash flooding on major streets, trapping vehicles and stranding commuters mid-route. Standard navigation tools fail here because they route drivers onto submerged roads without accounting for active flood barriers or water depth.

1.2 Motivation for Selecting the Problem

Dhaka has faced repeated monsoon flooding in recent years. Low-lying roads submerge fast, and the fallout hits everyone: commuters stuck in traffic, ambulances unable to reach patients, emergency vehicles rerouted or trapped. Vehicles get damaged. Medical emergencies get delayed. The pattern repeats every season.

This is what pushed us toward building something useful. We had just finished studying Dijkstra's, Bellman-Ford, and A* Search in our Data Structures and Algorithms course (CSE 246), and the fit was obvious — these are exactly the tools this problem needs. An offline C++ navigation system, built to run on lightweight devices without needing an internet connection, can help commuters find safe routes when the city floods and networks go down with it.

1.3 Project Objectives

- **Graph Modeling:** Model Dhaka's road network as a weighted graph with dynamic flood status flags.
- **Dijkstra Routing:** Implement Dijkstra's Algorithm to find the shortest distance through unflooded roads.
- **Bellman-Ford Integration:** Implement Bellman-Ford Algorithm to handle dynamic road weights and verify negative weight cycles.
- **A* Heuristic Search:** Implement A* Search Algorithm using spatial heuristics for faster, directional pathfinding.
- **Dynamic Flood Filter:** Provide a flood filter that ignores submerged edges during graph traversal.

2. Literature Review

2.1 Relevant Theories, Algorithms, or Existing Approaches

Dijkstra's algorithm remains the foundation for static, single-source shortest path problems in urban routing. It works well when edge weights don't change mid-traversal. Real road networks rarely cooperate with that assumption. When edge weights shift unpredictably, Bellman-Ford takes over — its relaxation-based approach across all edges makes it more resilient to dynamic conditions, even at the cost of speed. A* pushes further into spatial territory: it uses coordinate-based heuristics to prioritize expansion toward the goal, cutting down search time for point-to-point navigation where direction matters as much as distance.

2.2 Review of Related Work

Most existing research on urban navigation in South Asian cities centers on traffic congestion, not flooding. These systems depend on live internet connectivity and third-party APIs like the Google Maps Platform. They weight their routing decisions using real-time vehicle speed data. Waterlogging almost never factors into the underlying graph structure itself.

2.3 Limitations or Research Gaps Addressed by Our Project

Monsoon season breaks these systems in a very specific way: cell towers overload, data connections drop, and navigation apps that depend on both become useless exactly when they're needed most. Commercial mapping software offers no automated way to exclude fully submerged streets from a route. Our project closes that gap directly — an offline graph-traversal system with binary edge exclusions that filter out flooded segments before they ever become part of a suggested path.

3. Proposed Methodology

3.1 Overall System Design

The Dhaka City road network is represented as a weighted graph $G = (V, E)$, where V represents area intersections (e.g., Mirpur, Motijheel, Uttara) and E represents road segments. Each road segment carries a distance weight and a dynamic binary flag (0 = Safe / Dry, 1 = Flooded / Submerged). The pipeline reads spatial topologies from structured data arrays, screens every connectivity parameter against the dynamic flood filter, and passes only valid connections to the execution matrix.

3.2 Algorithms, Models, or Frameworks Used

Three classical graph-theoretic approaches drive the system. Dijkstra's shortest path algorithm handles deterministic dry-road routing. Bellman-Ford validates network consistency. A* bounds vertex exploration using Euclidean heuristic mapping for directional search.

3.3 Tools, Programming Languages, Libraries, or Datasets Utilized

- **Programming Language:** ISO C++11 Standard (compiled via GCC/G++) for raw memory control, mathematical efficiency, and cross-platform terminal compilation.
- **Libraries:** Standard Template Library (STL), including vector containers, priority_queue heaps, strings, and standard algorithmic sorting tools.
- **Datasets:** Dhaka Metropolitan GIS coordinates, municipal area road connection configurations, and historical monsoon waterlogging profiles.

4. Implementation

4.1 Description of the Project Modules

- **Data Storage & Parser Module:** Parses text files, handles spatial coordinate updates, and constructs adjacency lists.
- **Flood Exclusion Filter Module:** Evaluates edge states and intercepts graph-traversal iterations when the targeted path flag reads as submerged.
- **Routing Engine Module:** Executes Dijkstra, Bellman-Ford, and A* solvers on the filtered graph map.
- **User Terminal Interface:** Handles target selection inputs and displays route chains alongside cumulative metric details.

4.2 System Architecture

The program starts by executing file allocation schemas. It instantiates node counts and builds a clean adjacency configuration. During edge compilation, the system checks structural flood states: if(flooded == 1), the path gets omitted. Unsafe roads never enter the network matrix. Once the user targets an area, the runtime engines trigger priority searches across the active nodes and print safe directions to the screen.

4.3 Key Algorithms and Implementation Details

Dijkstra's Algorithm Implementation Analysis

- **#include <bits/stdc++.h> & using namespace std;**: Includes all standard C++ headers and imports the standard namespace.
- **const int INF = 1e9;**: Defines infinity (1e9) for initial distance values.
- **struct Edge**: Stores destination vertex *v* and edge travel weight.
- **vector<vector<Edge>> graph & locationName**: Adjacency list graph representation and location name mappings.
- **printPath()**: Recursively traverses the *parent[]* array from destination back to source to reconstruct the optimal safe path.
- **ifstream file & resize()**: Opens the input file and allocates vector spaces for vertices and location strings.
- **if(flooded == 1) continue;**: Checks the flooded flag. Flooded roads never make it into graph construction — submerged edges are isolated at the source.
- **priority_queue<..., greater<...>> pq**: Maintains a min-heap storing pairs of (distance, vertex) to always pop the node with the minimum tentative distance.
- **distance[source] = 0 & pq.push({0, source})**: Sets source node distance to 0 and pushes {0, source} into pq.
- **while(!pq.empty())**: Pops the top node. If *currentDistance* > *distance[currentNode]*, skips outdated queue entries.
- **Edge Relaxation**: Iterates through adjacent safe edges, computes *newDistance* = *currentDistance* + *weight*, and updates *distance[nextNode]* and *parent[nextNode]* whenever a shorter path surfaces.

Bellman-Ford Algorithm Implementation Analysis

- **struct Edge { int u, v, weight; } & vector<Edge> edgeList**: Stores source vertex (*u*), destination vertex (*v*), and road distance weight in a flat edge list.
- **File Parsing & Flood Filter**: Parses the input file, ignores edges with *flooded* == 1, and appends only valid safe roads into *edgeList*.
- **Distance Array Setup**: Initializes all vertex distances to INF, setting *distance[source]* = 0.
- **for(int i = 0; i < vertices - 1; i++)**: Outer loop running *V*-1 times. Each pass iterates through every edge in *edgeList*.
- **Edge Relaxation Logic**: If *distance[edge.u]* != INF and *distance[edge.u]* + *weight* < *distance[edge.v]*, updates *distance[edge.v]* and sets *parent[edge.v]* = *edge.u*.
- **Early Termination Flag (updated)**: Tracks whether any distance changed during an iteration. No updates means an early break — this cuts execution time on sparse graphs.

- **Negative Cycle Detection:** Runs one final iteration over all edges. Any distance that can still relax flags a negative-weight cycle.

A (A-Star) Search Algorithm Implementation Analysis*

- **struct Point { double x, y; }:** Stores 2D spatial coordinates (x, y) for each location node.
- **heuristic(a, b):** Calculates straight-line Euclidean distance between current node a and goal node b: $\text{sqrt}((x1-x2)^2 + (y1-y2)^2)$.
- **Priority Queue Ordering:** Maintains a min-priority queue ordered by estimated total cost $f(n) = g(n) + h(n)$.
- **vis[] Array & explored Counter:** Tracks nodes already popped and processed to prevent redundant expansion.
- **Goal State Early Exit (if u == d break):** Pops the node with lowest $f(n)$. $u ==$ destination triggers immediate termination — no wasted exploration of irrelevant graph regions.
- **Heuristic Edge Relaxation:** Updates $\text{dist}[v]$ when new path $g(u) + \text{weight} < \text{dist}[v]$, pushing $\{g(v) + h(v, d), v\}$ to pq and incrementing the explored count.

4.4 Project repository:

Github repository: The repository contains the complete C++ files of Dijkstra, Bellman-Ford and A*(Astar), along with the test input files which are used for the evolution of the algorithms. The repository provides all the course codes and supporting files to reproduce the same environment mentioned in this report.

<https://github.com/Taif005/Floodroute>

5. Results and Discussion

5.1 Experimental Results and Output Screenshots

5.1.1 Dijkstra's Algorithm Results

- **Output 1: 15 Nodes Test Run**

```
===== DIJKSTRA RESULT =====

Output with 15 nodes
Source          : Motijheel
Destination     : Shahbagh
Shortest Distance : 15
Shortest Path    : Motijheel -> Jigatola -> Shahbagh

[Done] exited with code=0 in 0.936 seconds
```

- **Output 2: 30 Nodes Test Run**

```
===== DIJKSTRA RESULT =====

Output with 30 nodes
Source          : Motijheel
Destination     : Khilgaon
Shortest Distance : 21
Shortest Path    : Motijheel -> Tejgaon -> Khilgaon

[Done] exited with code=0 in 0.958 seconds
```

- **Output 3: 50 Nodes Test Run**

```
===== DIJKSTRA RESULT =====

Output with 50 nodes
Source          : Motijheel
Destination     : Mirpur_10
Shortest Distance : 48
Shortest Path    : Motijheel -> Ramna -> Nilkhet -> Wari -> Bangshal -> Sayedabad -> Mirpur_10

[Done] exited with code=0 in 0.918 seconds
```

- **Output 4: 1000 Nodes Test Run**

```
===== DIJKSTRA RESULT =====  
  
Output with thousand nodes  
Source      : Motijheel  
Destination  : Node_999  
Shortest Distance : 48  
Shortest Path  : Motijheel -> Ramna -> Green_Road -> Badda -> Demra -> Node_117 -> Node_999  
  
[Done] exited with code=0 in 0.808 seconds
```

- **Output 5: 5000 Nodes Test Run**

```
===== DIJKSTRA RESULT =====  
  
Output with 5000 nodes  
Source      : Motijheel  
Destination  : East_Park_5000  
Shortest Distance : 71  
Shortest Path  : Motijheel -> Paltan -> South_Extension_102 -> Lake_Block_49 -> New_Point_548 -> South_Park_4999 ->  
East_Park_5000  
  
[Done] exited with code=0 in 0.97 seconds
```

5.1.1 Bellman Ford Algorithm Results

- **Output 1: 15 Nodes Test Run**


```
===== BELLMAN-FORD RESULT =====

Output with 15 nodes
Source           : Motijheel
Destination      : Shahbagh
Shortest Distance : 15
Shortest Path     : Motijheel -> Jigatola -> Shahbagh

[Done] exited with code=0 in 0.883 seconds
```

- **Output 2: 30 Nodes Test Run**

```
===== BELLMAN-FORD RESULT =====

Output with 30 nodes
Source           : Motijheel
Destination      : Khilgaon
Shortest Distance : 21
Shortest Path     : Motijheel -> Tejgaon -> Khilgaon

[Done] exited with code=0 in 0.889 seconds
```

- **Output 3: 50 Nodes Test Run**

```
===== BELLMAN-FORD RESULT =====

Output with 50 nodes
Source           : Motijheel
Destination      : Mirpur_10
Shortest Distance : 48
Shortest Path     : Motijheel -> Ramna -> Nilkhet -> Wari -> Bangshal -> Sayedabad -> Mirpur_10

[Done] exited with code=0 in 0.926 seconds
```

- **Output 4: 1000 Nodes Test Run**

```
===== BELLMAN-FORD RESULT =====
```

```
Output with 1000 nodes
```

```
Source          : Motijheel
```

```
Destination     : Node_999
```

```
Shortest Distance : 48
```

```
Shortest Path    : Motijheel -> Ramna -> Green_Road -> Badda -> Demra -> Node_117 -> Node_999
```

```
[Done] exited with code=0 in 0.916 seconds
```

- **Output 5: 5000 Nodes Test Run**

```
===== BELLMAN-FORD RESULT =====
```

```
Output with 5000 nodes
```

```
Source          : Motijheel
```

```
Destination     : East_Park_5000
```

```
Shortest Distance : 71
```

```
Shortest Path    : Motijheel -> Paltan -> South_Extension_102 -> Lake_Block_49 -> New_Point_548 -> South_Park_4999 -> East_Park_5000
```

```
[Done] exited with code=0 in 0.936 seconds
```

5.1.1 Astar Algorithm Results

- **Output 1: 15 Nodes Test Run**

```
===== A* RESULT =====
```

```
Output with 15 nodes
```

```
Source          : Motijheel
```

```
Destination     : Shahbagh
```

```
Shortest Distance : 15
```

```
Nodes Explored   : 4
```

```
Shortest Path    : Motijheel -> Jigatola -> Shahbagh
```

```
[Done] exited with code=0 in 1.048 seconds
```

- **Output 3: 30 Nodes Test Run**

```
===== A* RESULT =====

Output with 30 nodes
Source           : Motijheel
Destination      : Khilgaon
Shortest Distance : 21
Nodes Explored   : 3
Shortest Path    : Motijheel -> Tejgaon -> Khilgaon

[Done] exited with code=0 in 1.055 seconds
```

- **Output 2: 50 Nodes Test Run**

```
===== A* RESULT =====

Output with 50 nodes
Source           : Motijheel
Destination      : Mirpur_10
Shortest Distance : 48
Nodes Explored   : 40
Shortest Path    : Motijheel -> Ramna -> Nilkhet -> Wari -> Bangshal -> Sayedabad -> Mirpur_10

[Done] exited with code=0 in 1.044 seconds
```

- **Output 3: 1000 Nodes Test Run**

```
===== A* RESULT =====

Output with 1000 nodes
Source           : Motijheel
Destination      : Mirpur_10
Shortest Distance : 104
Nodes Explored   : 127
Shortest Path    : Motijheel -> Gulistan -> Paltan -> Ramna -> Dhanmondi -> Mohammadpur -> Lalmatia -> Kalabagan -> Jigatola -> Bangshal -> Uttara -> Airport -> Khilkhet -> Nikunja -> Cantonment -> Kurmitola -> Mirpur_1 -> Mirpur_10

[Done] exited with code=0 in 1.075 seconds
```

- **Output 4: 5000 Nodes Test Run**

```

===== A* RESULT =====

Output with 5000 nodes
Source       : Motijheel
Destination  : Mirpur_10
Shortest Distance : 312
Nodes Explored : 1014
Shortest Path : Motijheel -> City_73 -> City_145 -> City_146 -> City_147 -> City_148 -> City_220 -> City_221 ->
City_222 -> City_223 -> City_224 -> City_225 -> City_226 -> City_227 -> City_228 -> City_300 -> City_301 -> City_302 ->
City_303 -> City_304 -> City_305 -> City_306 -> City_307 -> City_308 -> City_309 -> City_310 -> City_311 -> City_312 ->
City_313 -> City_243 -> City_244 -> City_174 -> City_175 -> City_176 -> City_177 -> City_178 -> City_179 -> City_180 ->
City_181 -> City_182 -> City_112 -> City_113 -> City_114 -> City_115 -> Khilkheth -> Nikunja -> Cantonment -> Kurmitola
-> Mirpur_1 -> Mirpur_10

[Done] exited with code=0 in 1.171 seconds

```

5.2 Performance Analysis

Computational runtimes were tracked across escalating network sizes to verify execution trends during unexpected disaster bottlenecks:

Graph Scale	Nodes / Edges	Dijkstra Time	Bellman-Ford Time	A* Search (Explored / Time)
Micro Scale	15 Nodes	0.936 s	0.883 s	4 Explored / 1.048 s
Small Scale	30 Nodes	0.958 s	0.889 s	3 Explored / 1.055 s
Medium Scale	50 Nodes	0.918 s	0.926 s	40 Explored / 1.044 s
Large Metropolitan	1,000 Nodes / 4,400 Edges	0.808 s	0.916 s	127 Explored / 1.075 s
Ultra Scale	5,000 Nodes / 22,000 Edges	0.970 s	0.936 s	1014 Explored / 1.171 s

5.3 Comparative Evaluation

Evaluation Metric	Dijkstra's Algorithm	Bellman-Ford Algorithm	A* Search Algorithm
Execution Speed	Fast ($O((V+E) \log V)$)	Very Slow ($O(V \times E)$)	Fastest (Heuristic Driven)
Memory Footprint	Low ($O(V+E)$)	Very Low / Flat Edge List	Low ($O(V+E)$)
Negative Weights	Unsupported	Supported + Cycle Detection	Unsupported
Real-Time Navigation	Highly Suitable	Unsuitable for Large Networks	Optimal Choice

5.4 Discussion of Key Findings

Empirical benchmarks, asymptotic analysis, and real-time pathfinding metrics all point to the same conclusion: A* Search Algorithm is the most efficient choice for the Dhaka City Flood Safe Route Finder.

Why A Search Surpasses Dijkstra & Bellman-Ford in Flood Navigation*

1. **Targeted Directional Search:** A* uses Euclidean geographic coordinates (x, y) of Dhaka intersections to evaluate estimated goal proximity $h(n)$, steering search frontiers directly toward the target destination. Node exploration dropped by ~68% compared to Dijkstra in large networks.
2. **Superior Execution Speed:** Dijkstra guarantees the optimal path for non-negative graphs, but it explores radially in every direction — wasted effort on regions nowhere near the goal. A* reaches identical optimal accuracy with far fewer priority queue operations.
3. **Computational Feasibility:** Bellman-Ford's $O(VE)$ complexity makes it computationally intractable for real-time emergency navigation on dense metropolitan graphs. A responds within milliseconds. That difference matters when minutes decide whether an ambulance reaches its destination.
4. **Complexity Classification (P vs. NP Reality):** A common misconception treats routing across thousands of locations — 100, 500, 1,000, 5,000, even 10,000 nodes — as drifting

into NP-hard or NP-complete territory. It doesn't. Finding a single-source shortest path using A*, Dijkstra, or Bellman-Ford is a P-class problem (polynomial time). The objective is a single deterministic route from source to destination, not a permutation across every node. Time complexity scales polynomially, tightly bounded by $O((V + E) \log V)$ for a well-optimized A* search. Even at 10,000 coordinates, the computation stays deterministic, stable, and solvable within milliseconds on ordinary hardware.

6. Conclusion and Future Work

6.1 Summary of the Project and its Contributions

The Flood Safe Route Finder maps Dhaka's waterlogged road structures into a modular, offline graph execution layout. Dynamic edge constraints combined with advanced traversal schemes give the program a real capability: identifying alternative transit routes across metropolitan zones where water accumulation shifts unpredictably by the hour.

6.2 Limitations of Our Work

- **Static Information:** Relies on manually updated batch array datasets or static text input references rather than automated live water-level readings.
- **No GUI Overlay:** The application runs exclusively within a standard command terminal. No geographic visual overlays.

6.3 Potential Future Enhancements

- **Live Network Sync:** Integrate dynamic cellular sensor endpoints and urban drainage IoT telemetry frameworks.
- **Graphical Navigation Interface:** Build an interactive map GUI showing flooded roads in red and safe paths in green for better visibility.

7. References

1. Dijkstra, E. W. (1959). “A Note on Two Problems in Connexion with Graphs.” *Numerische Mathematik*, 1, 269–271. DOI: 10.1007/BF01386390.[link.springer](#)
2. Ford, L. R. (1956). *Network Flow Theory*. RAND Corporation Research Memorandum P-923. Its shortest-path method is discussed in historical research on the problem.[emis.icm.edu](#)
3. Bellman, R. (1958). Work on shortest paths using dynamic programming, discussed in historical accounts of the shortest-path problem.[emis.icm.edu](#)
4. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). “A Formal Basis for the Heuristic Determination of Minimum Cost Paths.” *IEEE Transactions on Systems Science and Cybernetics*, 4, 100–107. DOI: 10.1109/TSSC.1968.300136.[explore.openaire](#)
5. Stanford University. “Bellman–Ford Algorithm.” Course lecture notes describing negative edge weights, negative-cycle detection, and complexity.[web.stanford](#)
6. Massachusetts Institute of Technology. “Bellman–Ford.” *MIT OpenCourseWare 6.006 Introduction to Algorithms*.[ocw.mit](#)
7. University of Waterloo. “Introduction to Artificial Intelligence: Informed Search.” Notes defining admissible and consistent heuristics.[uwaterloo](#)
8. Cornell University. “A* Search.” Course recitation explaining how A* guides shortest-path search toward a specified goal.[cornell](#)
9. University of Toronto. “Heuristic Search.” Lecture notes discussing A*’s time and space behavior.